UNIT 5 · TOPIC 5.3

Protecting Stored Data with Cryptography

Assignment, Answer Key

Teacher answer key, 27 points total.

Practice Questions

5.3.a · Encryption vs. hashing

1. An engineer needs to store credit card numbers so they can be processed and charged at the end of every month. Which cryptographic primitive must they use? (1 pt)

- **● A. Encryption, because the process must be reversible to retrieve the original card numbers**
- ○ B. Hashing, because it protects the integrity of the credit card numbers
- ○ C. Encoding, because Base64 keeps card numbers safe on the disk
- ○ D. Obfuscation, because it hides details and is fully reversible without keys

Why: Since the credit card numbers must be retrieved in their original form to charge them, the control must be reversible. This requires encryption, not hashing.

2. What is the primary security goal of adding a salt to a password before hashing it? (1 pt)

- ○ A. To make the hashing function compute slower and consume more CPU
- **● B. To ensure that identical passwords yield different digests, defeating precomputed lookup attacks**
- ○ C. To encrypt the password so it can be decrypted by administrators
- ○ D. To detect if the password has been modified during transmission over the network

Why: Salting ensures that identical passwords do not hash to the same digest. This makes precomputed rainbow tables useless because an attacker would have to compute a new table for each unique salt.

3. Select ALL of the following statements that represent common security misconceptions. (3 pts)

- **■ A. Encoding data in Base64 provides confidentiality because the output is unreadable.**
- **■ B. MD5 is safe for general password hashing as long as you do not use a salt.**
- □ C. If an attacker gets access to the database of hashed passwords, they cannot reverse the digests to recover the plaintext.
- **■ D. Hashing is a reversible process if you possess the master decryption key.**

Why: Base64 encoding is trivially reversible and provides zero security. MD5 is deprecated and vulnerable to collisions. Hashing is mathematically one-way and has no decryption key.

4. What is the term for the situation when two different inputs to a hash function produce the exact same digest output? (1 pt)

Acceptable answers: collision; hash collision; collisions

Why: A collision occurs when two distinct inputs yield the same hash digest, which undermines integrity guarantees.

5.3.b · Symmetric encryption and key management

5. A developer encrypts a bitmap image using AES in ECB mode. Even though AES itself is uncompromised, the ciphertext still reveals the outline of the original image. Why? (1 pt)

- **● A. ECB mode encrypts identical plaintext blocks into identical ciphertext blocks, leaking patterns in the data**
- ○ B. AES-128 keys are too short to hide image data

○ C. Bitmap files cannot be encrypted with any block cipher

○ D. ECB mode does not actually encrypt the first block

Why: ECB mode encrypts each block independently with the same key, so identical plaintext blocks always produce identical ciphertext blocks, exposing patterns like image outlines.

6. Select ALL statements that correctly describe Galois/Counter Mode (GCM). (3 pts)

■ A. GCM is an Authenticated Encryption with Associated Data (AEAD) mode

■ B. GCM provides both confidentiality and integrity verification via an authentication tag

□ C. GCM is functionally identical to ECB mode with a different name

■ D. If an attacker tampers with GCM ciphertext, decryption fails

Why: GCM is an AEAD mode: it encrypts (confidentiality) and produces an authentication tag (integrity), so tampering causes decryption/verification to fail. It is nothing like ECB.

7. A DEK (Data Encryption Key) needs to be stored long-term alongside the encrypted data it protects. Per the key-management lifecycle, what should be done with the DEK before storing it? (1 pt)

○ A. Store it in plaintext next to the ciphertext for fast access

● B. Wrap (encrypt) it with a Key Encryption Key (KEK) kept in an HSM or secrets manager, and store the wrapped DEK

○ C. Hardcode it into the application source code

○ D. Email it to the database administrator for safekeeping

Why: Envelope encryption wraps the DEK with a KEK; the wrapped DEK can be stored safely next to the data, while the KEK stays protected in an HSM or secrets manager.

8. Name the type of random number generator that MUST be used to generate a symmetric key, instead of a standard library like Python's `random` module. (1 pt)

Acceptable answers: CSPRNG; cryptographically secure pseudo-random number generator; cryptographically secure prng

Why: Keys must be generated with a CSPRNG (e.g., Python's `secrets` module or /dev/urandom) because standard PRNGs are predictable and unsuitable for cryptography.

5.3.c · Integrity and authenticity (HMAC)

9. An attacker intercepts a message and its plain SHA-256 hash in transit. They change the message and recompute the hash over the new content before forwarding both to the recipient. What happens when the recipient checks integrity? (1 pt)

● A. The check passes cleanly, because the attacker can compute a correct hash over any message they choose

○ B. The check fails, because SHA-256 detects tampering automatically

○ C. The check fails, because hash functions require a secret key to compute

○ D. The recipient cannot compute a hash at all without the original message

Why: Plain hash functions are public and keyless. An attacker can recompute a valid hash over their tampered message, so the recipient's check passes even though the message was altered. A plain hash proves the data matches its digest, not who produced that pairing.

10. What is the key structural weakness that makes naive `H(key || message)` vulnerable to a length-extension attack on a Merkle-Damgard hash like SHA-256? (1 pt)

● A. The digest exposes the hash function's internal state, letting an attacker resume hashing and append attacker-chosen data without knowing the key

○ B. The hash function is too slow to compute over long messages

○ C. SHA-256 always produces the same digest for every input

○ D. Concatenating the key and message causes a buffer overflow

Why: Merkle-Damgard hashes output their exact internal state as the digest. Given H(key||message), an attacker can resume hashing from that state to compute a valid hash for key||message||padding||attacker_data, without ever learning the key.

11. Select ALL statements that correctly describe HMAC. (3 pts)

■ A. HMAC nests the hash function twice, folding the key into both an inner and outer keyed input

■ B. HMAC is standardized in NIST FIPS 198-1

■ C. HMAC is provably resistant to length-extension attacks even when built on SHA-256

□ D. HMAC encrypts the message so only the key holder can read its contents

Why: HMAC(key, message) = H((key XOR opad) || H((key XOR ipad) || message)), a double nesting with two independently-derived keyed prefixes, standardized in FIPS 198-1, which defeats length-extension attacks. HMAC does not provide confidentiality; it produces a verification tag only, the message itself is not hidden.

12. Give the one-word term for the property that a plain hash provides (data was not altered) but does NOT prove on its own: that data really came from a specific, key-holding sender. (1 pt)

Acceptable answers: authenticity; authentication

Why: Integrity means the data was not altered. Authenticity means it really came from who it claims to, which requires a secret the attacker does not have, hence HMAC's keyed construction.

Coding Assignments (Python 3)

5.3.a · Encryption vs. hashing

3 pts · auto-graded on the site

Coding problem, 5.3.a: Encryption vs. hashing

Skill: Mitigate Risk · **Maps to:** 5.3.A · **Language:** Python 3 (pytest)

Problem

In secure systems, credentials like passwords should never be stored in their original plaintext form, nor should they be encrypted (which implies they could be decrypted if the key is compromised). Instead, they should be hashed using a one-way cryptographic function combined with a unique, random value called a **salt**. The salt ensures that identical passwords result in different digests, defeating precomputed lookup attacks (like rainbow tables).

Your task is to implement a password hashing and verification module.

You must implement the function `solve(action: str, password: str, param: str)` to handle two actions: 1. **Hash a password:** Given action `"hash"`, a plaintext password, and a salt, compute the SHA-256 hash of the concatenated string `salt + password`. Return a combined string of the format `salt$digest` (where `digest` is the hex-encoded hash). 2. **Verify a password:** Given action `"verify"`, a plaintext password, and a stored hash string in the format `salt$digest`, extract the salt and digest, compute the salted hash of the password, and verify if it matches the expected digest. Use a constant-time comparison helper like `hmac.compare_digest` to prevent timing side-channel attacks.

Function contract

Implement `solve(action: str, password: str, param: str) -> Union[str, bool]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

- `action`: Either `"hash"` or `"verify"`
- `password`: The plaintext password
- `param`: Either the `salt` (if hashing) or the `stored_hash` of format `salt$digest` (if verifying)

Output

- If hashing: A string of format `salt$digest`
- If verifying: A boolean indicating whether the password matches the digest

Examples

```
solve("hash", "myPassword", "somesalt")
# -> "somesalt$4f24ef3001859bf9016e7a750ef90333d833890ea70c99a6cfbf6e568ea46a9a"

solve("verify", "myPassword", "somesalt$4f24ef3001859bf9016e7a750ef90333d833890ea70c99a6cfbf6e568ea46a9a")
# -> True

solve("verify", "wrongPassword", "somesalt$4f24ef3001859bf9016e7a750ef90333d833890ea70c99a6cfbf6e568ea46a9a")
# -> False
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
import hashlib
import hmac
from typing import Union


def solve(action: str, password: str, param: str) -> Union[str, bool]:
    if action == "hash":
        salt = param
        hasher = hashlib.sha256()
        hasher.update((salt + password).encode("utf-8"))
        digest = hasher.hexdigest()
        return f"{salt}${digest}"
    elif action == "verify":
        stored_hash = param
        if "$" not in stored_hash:
            return False
        parts = stored_hash.split("$", 1)
        if len(parts) != 2:
            return False
        salt, expected_digest = parts
        hasher = hashlib.sha256()
        hasher.update((salt + password).encode("utf-8"))
        computed_digest = hasher.hexdigest()
        return hmac.compare_digest(computed_digest, expected_digest)
```

```
    else:
        raise ValueError("Invalid action. Must be 'hash' or 'verify'.")
```

5.3.b · Symmetric encryption and key management

3 pts · auto-graded on the site

Coding problem, 5.3.b: Symmetric key-management toolkit

Skill: Mitigate Risk · **Maps to:** 5.3.A, 5.3.B · **Language:** Python 3 (pytest)

Problem

Build three small helpers a security engineer would use when reasoning about symmetric key management, without relying on any real crypto library (standard library only, this is a toy, pedagogical model, not production cryptography).

1. **Key wrapping (envelope encryption).** Implement `wrap_key(dek, kek)` and `unwrap_key(wrapped, kek)` using a repeating-XOR keystream: each byte of dek is XORed with the KEK byte at the same position modulo `len(kek)`. This models how a Key Encryption Key (KEK) wraps a Data Encryption Key (DEK) for storage (see atom.md, "Key wrapping (Envelope Encryption)"). Unwrapping with the same KEK must recover the original DEK exactly, since XOR is its own inverse. 2. **Weak-RNG detection.** Implement `looks_like_csprng(key_bytes)`, a heuristic that flags keys that could NOT plausibly have come from a CSPRNG because they show low entropy / obvious structure (e.g., a single repeated byte, a short repeating pattern, or a strictly monotonic/arithmetic byte sequence like `0,1,2,3...`). Real CSPRNG output has no such structure. 3. **ECB pattern-leakage detection.** Implement `detect_ecb_leakage(ciphertext_blocks)`, which takes a list of same-size `bytes` blocks (as an ECB-mode block cipher would produce) and returns `True` if any two blocks are identical, the tell-tale sign that ECB mode is leaking repeated plaintext patterns into the ciphertext (see atom.md, "Electronic Codebook (ECB) Mode (Insecure)").

Function contract

Implement all three functions in `starter.py` (see `interface.md` / `blackbox.yaml`):

```
def wrap_key(dek: bytes, kek: bytes) -> bytes
def unwrap_key(wrapped: bytes, kek: bytes) -> bytes
def looks_like_csprng(key_bytes: bytes) -> bool
def detect_ecb_leakage(ciphertext_blocks: list) -> bool
```

Input

- `wrap_key`/`unwrap_key`: dek/wrapped are `bytes` (may be empty); kek is non-empty `bytes`.
- `looks_like_csprng`: `key_bytes` is `bytes`, at least 1 byte.
- `detect_ecb_leakage`: `ciphertext_blocks` is `list[bytes]`, all blocks the same length.

Output

- `wrap_key`/`unwrap_key` return `bytes` the same length as their key argument.
- `looks_like_csprng` returns `bool`.
- `detect_ecb_leakage` returns `bool`.

Examples

```
wrap_key(b"\x01\x02\x03\x04", b"\xff\x00")
# -> bytes([0x01^0xff, 0x02^0x00, 0x03^0xff, 0x04^0x00]) == b"\xfe\x02\xfc\x04"

unwrap_key(wrap_key(dek, kek), kek) == dek   # always, for any non-empty kek

looks_like_csprng(bytes([5]*16))            -> False   # single repeated byte
looks_like_csprng(bytes(range(16)))         -> False   # arithmetic/monotonic sequence
```

```
looks_like_csprng(b"\x4e\x91\x0a\xd3\x77\xbb\x02\xf8")  -> True   # no obvious structure

detect_ecb_leakage([b"AAAAAAAA", b"BBBBBBBB", b"AAAAAAAA"]) -> True    # block 0 == block 2
detect_ecb_leakage([b"AAAAAAAA", b"BBBBBBBB", b"CCCCCCCC"]) -> False
```

Constraints

- Deterministic, offline, standard library only. No `import random`, no real crypto libraries (`cryptography`, `pycryptodome`, `hashlib`-based ciphers, etc.).
- `kek` is always non-empty for `wrap_key`/`unwrap_key`; `dek` may be empty (returns `b""`).
- `looks_like_csprng` receives at least 1 byte; treat any input shorter than 8 bytes as "too short to judge" and return `False` (not enough data to look random).
- `detect_ecb_leakage` receives zero or more equal-length `bytes` blocks.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
def wrap_key(dek: bytes, kek: bytes) -> bytes:
    if not kek:
        raise ValueError("kek must be non-empty")
    return bytes(b ^ kek[i % len(kek)] for i, b in enumerate(dek))


def unwrap_key(wrapped: bytes, kek: bytes) -> bytes:
    return wrap_key(wrapped, kek)


def looks_like_csprng(key_bytes: bytes) -> bool:
    n = len(key_bytes)
    if n < 8:
        return False

    if len(set(key_bytes)) == 1:
        return False

    for period in range(1, n // 2 + 1):
        pattern = key_bytes[:period]
        tiled = (pattern * (n // period + 1))[:n]
        if tiled == key_bytes:
            return False

    step = (key_bytes[1] - key_bytes[0]) % 256
    is_arithmetic = all(
        (key_bytes[i] - key_bytes[i - 1]) % 256 == step for i in range(1, n)
    )
    if is_arithmetic:
        return False

    return True


def detect_ecb_leakage(ciphertext_blocks: list) -> bool:
    seen = set()
    for block in ciphertext_blocks:
        if block in seen:
```

```
            return True
        seen.add(block)
    return False
```

5.3.c · Integrity and authenticity (HMAC)

3 pts · auto-graded on the site

Coding problem, 5.3.c: Implement HMAC-SHA256 by hand

Skill: Mitigate Risk · **Maps to:** 5.3.B, 5.3.C · **Language:** Python 3 (pytest)

Problem

An API needs to verify that requests were sent by a client holding a shared secret key, and that the request body wasn't tampered with in transit. Rather than calling Python's built-in `hmac` module, build the FIPS 198-1 HMAC construction yourself on top of `hashlib.sha256`, so you can see exactly how the ipad/opad key-nesting defeats length-extension attacks.

Function contract

Implement, in `starter.py`:

- `hmac_sha256(key: bytes, message: bytes) -> str`, return the HMAC-SHA256 tag as a lowercase 64-character hex string.
- `verify(key: bytes, message: bytes, tag: str) -> bool`, recompute the tag and compare it (case-insensitively) to the given `tag` using a constant-time comparison.
- `solve(data: dict) -> dict`, the blackbox entry point that dispatches to the two functions above (see `interface.md` / `blackbox.yaml`).

You must implement the ipad/opad key derivation and the double-nested hashing yourself. You may use `hashlib.sha256` only as the inner hash primitive `H()`, do not import the `hmac` module for the core logic.

The construction (FIPS 198-1)

```
HMAC(key, message) = H( (key XOR opad) || H( (key XOR ipad) || message ) )
```

- Block size for SHA-256 is 64 bytes.
- If `key` is longer than 64 bytes, replace it with `H(key)` first.
- Right-pad `key` with `0x00` bytes up to exactly 64 bytes -> call this `k0`.
- `ipad` = the byte `0x36` repeated 64 times; `opad` = the byte `0x5c` repeated 64 times.
- `inner_key = k0 XOR ipad`, `outer_key = k0 XOR opad` (byte-wise XOR).
- `inner_hash = H(inner_key || message)`
- final tag = `H(outer_key || inner_hash)`, hex-encoded.

Input / Output

- `hmac_sha256(key, message)`: `key: bytes`, `message: bytes` -> hex `str` tag.
- `verify(key, message, tag)`: `tag: str` (hex, any case) -> `bool`.

- `solve(data)`: `data = {"op": "tag", "key": "<hex>", "message": "<str>"}` -> `{"tag": "<hex>"}`, or `data = {"op": "verify", "key": "<hex>", "message": "<str>", "tag": "<hex>"}` -> `{"valid": <bool>}`.

Examples

```
hmac_sha256(b"secretkey", b"transfer $100 to acct 42")
  -> "5b1f9a2c...b3" (64 hex chars; must match Python's hmac.new(key, msg, sha256).hexdigest())

verify(b"secretkey", b"transfer $100 to acct 42", "5b1f9a2c...b3") -> True
verify(b"secretkey", b"transfer $900 to acct 42", "5b1f9a2c...b3") -> False  # tampered message
```

Constraints

- Deterministic, offline, standard library only (`hashlib` only, no `hmac` module).
- Correctness is checked against Python's own `hmac` module inside `tests.py` (used there only as an independent oracle, not as part of your implementation).
- Must pass all cases in `tests.py` (visible + hidden), including empty keys/messages, keys longer than the 64-byte block size, and case-insensitive tag comparison in `verify()`.

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
import hashlib

BLOCK_SIZE = 64
IPAD = 0x36
OPAD = 0x5C


def _h(data: bytes) -> bytes:
    return hashlib.sha256(data).digest()


def _derive_key(key: bytes) -> bytes:
    if len(key) > BLOCK_SIZE:
        key = _h(key)
    return key + b"\x00" * (BLOCK_SIZE - len(key))


def _xor_bytes(data: bytes, pad: int) -> bytes:
    return bytes(b ^ pad for b in data)


def hmac_sha256(key: bytes, message: bytes) -> str:
    k0 = _derive_key(key)
    inner_key = _xor_bytes(k0, IPAD)
    outer_key = _xor_bytes(k0, OPAD)
    inner_hash = _h(inner_key + message)
    tag = _h(outer_key + inner_hash)
    return tag.hex()


def _constant_time_equal(a: str, b: str) -> bool:
    if len(a) != len(b):
        return False
    result = 0
```

```
    for ca, cb in zip(a, b):
        result |= ord(ca) ^ ord(cb)
    return result == 0


def verify(key: bytes, message: bytes, tag: str) -> bool:
    expected = hmac_sha256(key, message)
    return _constant_time_equal(expected, tag.lower())


def solve(data: dict) -> dict:
    op = data["op"]
    key = bytes.fromhex(data["key"])
    message = data["message"].encode("utf-8")
    if op == "tag":
        return {"tag": hmac_sha256(key, message)}
    elif op == "verify":
        return {"valid": verify(key, message, data["tag"])}
    else:
        raise ValueError(f"unknown op: {op!r}")
```